

CESE5040 – High-Performance Computing and AI Architectures

Lab Assignment 1: Sequential analysis, vectorization and JIT

Daniel Tyukov
5714699

April 25, 2026

1 Student information

Name: Daniel Tyukov

Student number: 5714699

Notes.

- Provide your answers following the template made available in Brightspace.
- Provide your name and student number inside the report.
- Provide your student number on the file name.
- Provide your answers at least 3 days before the next lecture.
- When answering the questions, take all measurements using the same machine.

Hardware used for all measurements. All timing numbers in this report come from the course AWS VM: Intel Xeon Platinum 8259CL @ 2.50 GHz (Cascade Lake), 1 physical core / 2 SMT threads, 3.7 GiB RAM, Ubuntu 24.04, kernel 6.17.0-1010-aws, Python 3.12.3, NumPy 2.4.4, Numba 0.65.0, SciPy 1.17.1.

2 Lab Exercise 1

2.1 Question 1 (Exercise 1.1) – Pen-and-paper analysis

(a) **Time complexity.** The body of `simulate` has, per timestep $t > 0$, the structure:

```
for n in 0..N-1:                                # outer: N
    c_in = calculate_coupling(...)                # inner: N (sum over j)
    X_new = step(...)                             # O(cost(F))
    Xs[n][:][t] = X_new                          # O(M)
```

`calculate_coupling` performs N multiply-adds (one per source j). With M and $\text{cost}(F)$ both constant in this lab, the dominant per-timestep cost is the $N \times N$ coupling sum: $\mathcal{O}(N^2)$ per timestep. Across $T = \text{tf}/\text{dt}$ timesteps,

$$T_{\text{seq}} = \mathcal{O}(N^2 \cdot T) = \mathcal{O}\left(\frac{N^2 \cdot \text{tf}}{\text{dt}}\right).$$

Quadratic in N (dense coupling sum) and linear in T (the time loop is purely sequential due to the causal delay dependency, see (d)).

(b) Memory capacity. The state-history array is $\mathbf{Xs}[N][M][T]$, so the number of stored state elements is

$$\# \text{elements}(\mathbf{Xs}) = N \cdot M \cdot T = N \cdot M \cdot \frac{\mathbf{tf}}{\mathbf{dt}}.$$

Scaling individually with each parameter (the assignment writes “D”; read as $\text{Duration} = \mathbf{tf}$, the only un-itemised simulation parameter that does not have its own letter):

Table 1: Scaling of state memory with each simulation parameter.

Parameter	Scaling of state memory
N (number of centers)	linear, $\propto N$
M (state variables per center)	linear, $\propto M$
$D \equiv \mathbf{tf}$ (duration)	linear, $\propto \mathbf{tf}$ (since $T = \mathbf{tf}/\mathbf{dt}$)

Auxiliary structures (W , distances, integer delays) are each $N \times N$, i.e. $\mathcal{O}(N^2)$ but constant in time – sub-dominant to $\mathbf{Xs}$ once $T \gtrsim N/M$, which holds for all three TVB datasets at $\mathbf{tf} = 150$ ms. For the largest dataset (TVB998, $M = 2$, $T = 3000$) the state array is $N \cdot M \cdot T \approx 6 \times 10^6$ float64, ≈ 48 MB – comfortably within the VM’s 3.7 GiB RAM.

(c) Linear pre-synapse transformation. If $K_{\text{pre}}(X_j, X_i) = aX_j + bX_i$, the inner coupling sum factors:

$$\begin{aligned} \sum_{j=1}^N W_{ij} K_{\text{pre}}(X_j(t - d_{ij}), X_i(t)) &= \sum_{j=1}^N W_{ij} (a X_j(t - d_{ij}) + b X_i(t)) \\ &= a \sum_{j=1}^N W_{ij} X_j(t - d_{ij}) + b X_i(t) \sum_{j=1}^N W_{ij}. \end{aligned}$$

Define $S_i = \sum_{j=1}^N W_{ij}$. Because W is constant for the entire simulation, S_i can be *precomputed once* at initialisation and reused every timestep. The $b X_i(t)$ term then leaves the inner j -loop entirely.

Table 2: Operation count per destination i per timestep, before and after the factoring transformation.

Form	Multiplies in j -loop	Adds in j -loop	Outside j -loop
Original K_{pre} inline	$\approx 3N$	N	—
Factored	N	N	1 multiply, 1 add

The constant factor on the dominant multiply-add count drops by roughly $3\times$ in the inner loop, while the S_i precompute is a one-time $\mathcal{O}(N^2)$ cost (cheaper than even one timestep of the original loop). The asymptotic complexity stays $\mathcal{O}(N^2T)$, but the constant shrinks – relevant for any inner-loop acceleration (vectorization, JIT).

(d) Parallelism across timesteps. No, not in general. The Forward Euler update at step t reads $X_j(t - d_{ij})$ from history, with $d_{ij} \geq 0$. Whenever the script’s branch `if (i == n) or (D_row[i] == 0): x_src = Xs[i][0][t-1]` fires (self-loops or zero-delay edges), step t depends on step $t - 1$ – a hard data dependency that serialises the time loop.

Formally, let $d_{\min} = \min\{d_{ij} : W_{ij} \neq 0\}$. Steps $t, t + 1, \dots, t + d_{\min} - 1$ are eligible for parallel execution because each only reads from history $\leq t - 1$. For real TVB connectomes $d_{\min}$ is small (zero for self-loops in this code), so the practical parallel slack across timesteps is essentially *one* – i.e. no useful parallelism along the time axis.

Within a single timestep, by contrast, the N centers can be updated in parallel: all of them read from `Xs[*][:][ $\leq t-1$ ]` and write to `Xs[*][:][ $t$ ]`, so there are no read-after-write conflicts among them.

2.2 Question 2 (Exercise 1.2) – Profile the sequential implementation

`tvb_seq.py` is already instrumented with `tic-toc` pairs around the body of `simulate` (the `start/end` brackets) and around every `calculate_coupling` call (the `c_duration` accumulator). The driver script `bench_phase2.py` calls `tvb_seq.simulate` on each dataset in turn at `tf = 150 ms`, `dt = 0.05 ms` ($T = 3000$ timesteps).

Table 3: Execution times for `tvb_seq.py` over 150 timesteps on the course AWS VM.

Dataset	N	Total time (s)	<code>calculate_coupling</code> time (s)	Coupling fraction p
TVB76	76	5.051	4.831	95.65%
TVB192	192	26.728	26.032	97.40%
TVB998	998	628.744	624.182	99.27%

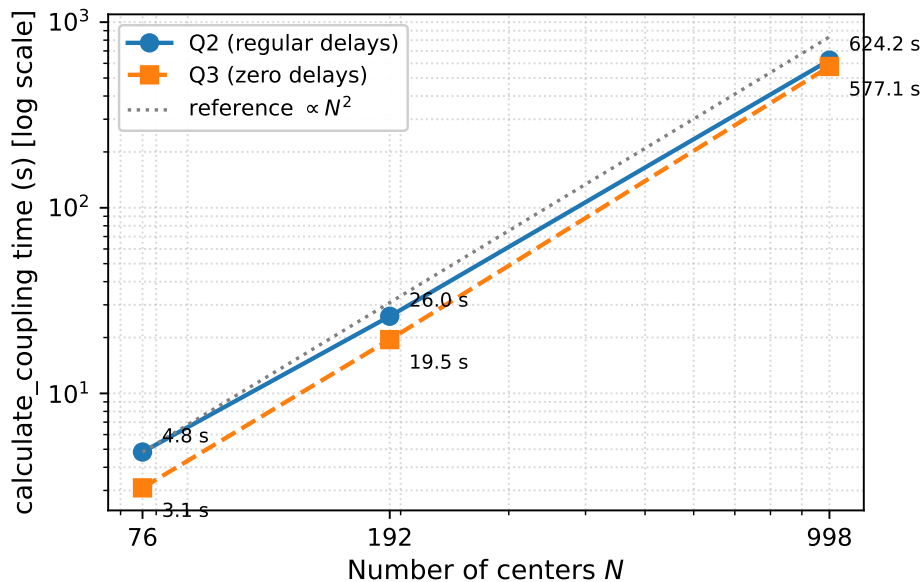


Figure 1: Coupling time vs. number of centers N , log–log axes. Q2 (regular delays) and Q3 (zero delays) both follow the dashed reference line $\propto N^2$ closely; the sub-quadratic shortfall is consistent with constant-time per-iteration overhead becoming a smaller share of total work as N grows.

Does it match the $\mathcal{O}(N^2T)$ prediction? The per- N scaling of the coupling time:

- TVB76 $\rightarrow$ TVB192: N ratio $192/76 = 2.53$, N^2 ratio 6.39. Measured coupling-time ratio $26.032/4.831 = 5.39$.
- TVB192 $\rightarrow$ TVB998: N ratio $998/192 = 5.20$, N^2 ratio 27.02. Measured coupling-time ratio $624.182/26.032 = 23.98$.

Both ratios are slightly sub-quadratic (about $0.84N^2$ and $0.89N^2$). The shortfall is consistent with constant-time per-iteration overheads in the Python interpreter (loop dispatch, attribute lookups) becoming a smaller share of total work as N grows – the inner multiply-add becomes more dominant. The data therefore confirms $\mathcal{O}(N^2T)$ to first order.

What fraction of execution time is in `calculate_coupling`? 95.65% at $N = 76$, climbing to 99.27% at $N = 998$. Coupling dominates, and its share grows with N .

Why? Per timestep, coupling does $N \cdot N$ work while local dynamics + Forward Euler does $\mathcal{O}(N \cdot M)$ work. The ratio $N^2/(N \cdot M) = N/M$ grows linearly with N , so any non-coupling overhead becomes asymptotically negligible.

2.3 Question 3 (Exercise 1.3) – Zero-delay variant

The same driver, but with `D` overridden to all zeros before calling `simulate`. The integer-delay matrix `D_timestep` is consequently all zero, so the guard `if t >= D_row[i]` always passes and the inner branch always takes the `(D_row[i] == 0) ⇒ Xs[i][0][t-1]` path.

Table 4: Execution times with zero distances, 150 timesteps. The two right-most columns express the Q3 timing as a saving relative to Q2.

Dataset	Total time (s)	<code>calculate_coupling</code> time (s)	Δ vs Q2 (total)	Q2 / Q3
TVB76	3.309	3.098	−1.74 s (−34.5%)	1.53×
TVB192	20.134	19.487	−6.59 s (−24.7%)	1.33×
TVB998	581.393	577.055	−47.35 s (−7.5%)	1.08×

Are there differences? Why? Yes – a measurable difference, which shrinks as N grows. Two mechanisms account for the speedup of Q3 over Q2:

1. **Branch elision.** With $D = 0$ the inner `if/else` collapses into a single branch: every (i, n) pair takes the `Xs[i][0][t - 1]` path. Q2’s mixed branching forces the interpreter through more `if`-tests per iteration.
2. **Memory locality.** All `Xs[i][0][t - 1]` reads land at the same “column” $t - 1$ of the state history. Even with Python lists, repeated access to the same nested structure is cheaper than scattered reads from `Xs[i][0][t - dij]`.

The gap shrinks with N because the branch-overhead saving is *constant* per inner iteration, while the actual multiply-add work *grows*. As N increases, the constant saving is diluted ($1.53\times \rightarrow 1.33\times \rightarrow 1.08\times$). By TVB998 the inner-loop body dominates and branching is in the noise.

2.4 Question 4 (Exercise 1.4) – Sparse (CSR) computation

(a) Theoretical speedup with sparsity $S\%$. In the dense version, `calculate_coupling` runs N multiply-adds per destination row i (one per source j , regardless of W_{ij}). The CSR version runs only NNZ_i multiply-adds, where NNZ_i is the number of non-zeros in row i . Letting $\text{NNZ} = \sum_i \text{NNZ}_i$ and using $\text{NNZ} = N^2(1 - S/100)$:

$$S_{\text{coupling}}^{\text{theory}} = \frac{N^2}{\text{NNZ}} = \frac{1}{1 - S/100}.$$

This applies *only* to `calculate_coupling`. The end-to-end speedup is bounded by Amdahl’s law,

$$S_{\text{end-to-end}} = \frac{1}{(1 - p) + p/S_{\text{coupling}}},$$

where p is the fraction of total runtime spent in `calculate_coupling` (measured in Q2 above).

(b) **TVB192 sparsity.** Computed from the cached connectome via `np.count_nonzero(W) / W.size`:

Table 5: Sparsity of all three TVB connectomes.

Dataset	N	total entries	non-zeros	zeros	sparsity $S\%$	density
TVB76	76	5,776	1,560	4,216	72.99%	27.01%
TVB192	192	36,864	3,532	33,332	90.42%	9.58%
TVB998	998	996,004	35,730	960,274	96.41%	3.59%

For TVB192, $S = 90.4188\%$, giving a theoretical coupling speedup of $1/(1-0.9042) \approx \mathbf{10.44\times}$.

(c) **Measured speedup vs. theoretical.** `tvb_seq_sparse.py`, TVB192, 150 timesteps:

Table 6: TVB192 sparse measurement compared to Q2 dense baseline.

Quantity	Value
<code>tvb_seq.py</code> (dense, Q2) total	26.728 s
<code>tvb_seq.py</code> (dense, Q2) coupling	26.032 s
<code>tvb_seq_sparse.py</code> total	3.847 s
<code>tvb_seq_sparse.py</code> coupling	3.323 s
Measured coupling speedup	$26.032/3.323 = \mathbf{7.83\times}$
Theoretical coupling speedup (a + b)	10.44×
Measured end-to-end speedup	$26.728/3.847 = 6.95\times$
Amdahl-predicted end-to-end ($p = 0.974$, $k = 7.83$)	$6.65\times$

The measured coupling speedup ($7.83\times$) falls about 25% short of the theoretical $10.44\times$. Three mechanisms account for the gap:

1. **CSR overhead.** The sparse loop adds two layers of indirection (`col_index[i]` to find the source node, `D_sparse[i]` for the delay), which in CPython are extra `__getitem__` calls per iteration. The dense loop indexes a 2-D array with the loop variable directly.
2. **Branch overhead.** The `if t >= D_sparse[i] and (col_index[i] == n) or (D_sparse[i] == 0)` tests are still executed per non-zero, adding constant cost the simple count $N^2 \rightarrow \text{NNZ}$ does not capture.
3. **CSR build cost.** A one-time $\mathcal{O}(N^2)$ scan to build `W_sparse`, `D_sparse`, `col_index`, `row_pointer` is included in the total. For 150 timesteps it amortises well, but it shifts a few percent of the work from the steady-state coupling loop.

The end-to-end Amdahl prediction matches the measurement within $\sim 5\%$ ($6.65\times$ predicted vs. $6.95\times$ measured). The slight surplus over Amdahl is consistent with the non-coupling part also running marginally faster in sparse mode (fewer cache thrashes through `D_timestep`), but the dominant story is exactly Amdahl.

(d) **Downsides of sparse computation.**

1. **Indirect indexing kills cache locality.** CSR coupling reads `Xs[col_index[i], 0, t - D_sparse[i]]`; the index arrays produce an essentially random gather pattern that misses L1/L2 caches more often than the dense version’s contiguous walk.
2. **Vectorization / SIMD becomes harder.** Modern CPUs and NumPy excel at contiguous, regular-shape arithmetic. A run of NNZ_i multiplies per row, with NNZ_i varying across i , breaks the regularity that lets a vector unit process many elements per instruction.

3. **Pre-processing cost.** Building the four CSR arrays is a one-time $\mathcal{O}(N^2)$ scan over W . For short simulations this eats into the speedup; for long ones it amortises.
4. **Storage overhead per non-zero.** CSR stores three numbers per non-zero (value, column index, plus a fraction of `row_pointer`). Storage is a net win only when sparsity exceeds roughly 60–70%.
5. **Load imbalance under parallelism.** Different rows have different NNZ_i ; under row-partitioned parallelism “fat” rows become stragglers. The dense version has uniform per-row work and balances trivially.

2.5 Question 5 (Exercise 1.5) – MLP local dynamics

Table 7: TVB192 MLP non-sparse, 15 timesteps.

Metric	Time (s)	Fraction of total
Total simulation	4.717	—
step (incl. MLP f)	2.771	58.74%
calculate_coupling	1.916	40.62%

(a) **Run `tvb_seq_mlp.py` (dense) for 15 timesteps on TVB192.** For Q6’s required comparison, sequential dense MLP timings on the other two datasets at the same `tf`:

Table 8: Sequential dense MLP timings on all three datasets, 15 timesteps.

Dataset	N	Total (s)	step (s)	calculate_coupling (s)
TVB76	76	1.104	0.884	0.211
TVB192	192	4.717	2.771	1.916
TVB998	998	66.983	12.410	54.418

The coupling fraction climbs from 19.1% at TVB76 to 81.2% at TVB998 – the MLP cost is $\mathcal{O}(N)$, the coupling cost is $\mathcal{O}(N^2)$, so for large N the picture from Q2 reasserts itself even with the heavier per-node f .

(b) **Maximum theoretical speedup from accelerating coupling alone.** Amdahl’s law: with coupling fraction p and coupling speedup k ,

$$S_{\text{end-to-end}}(k) = \frac{1}{(1-p) + p/k}.$$

The maximum end-to-end speedup is the limit $k \rightarrow \infty$:

$$S_{\text{max}} = \frac{1}{1-p}.$$

For TVB192 with $p = 0.4062$,

$$S_{\text{max}} = \frac{1}{1-0.4062} \approx \mathbf{1.68\times}.$$

With the MLP as local dynamics, even infinite coupling acceleration caps the end-to-end speedup at $1.68\times$ – accelerating only the coupling is *not* the right plan when f is heavy; the step itself must be accelerated too.

Table 9: TVB192 MLP sparse vs. dense, 15 timesteps.

Metric	Q5a (dense)	Q5c (sparse)	Speedup vs Q5a
<code>calculate_coupling</code>	1.916 s	0.219 s	8.75×
<code>step</code>	2.771 s	2.340 s	1.18×
Total simulation	4.717 s	2.584 s	1.83×

(c) **Sparse MLP run – comparison to Amdahl’s law. Amdahl prediction.** With $p = 0.4062$ and the measured coupling speedup $k = 8.75$:

$$S_{\text{predicted}} = \frac{1}{(1 - 0.4062) + 0.4062/8.75} = \frac{1}{0.5938 + 0.0464} \approx 1.56 \times .$$

Measured: 1.83×. **Predicted:** 1.56×. The measured value is about 17% above Amdahl’s prediction. This isn’t a violation of the law – it shows the implicit assumption (that the non-accelerated part stays exactly the same) does not hold here: the `step` time also drops from 2.771 s to 2.340 s (a 1.18× side-speedup, likely from reduced `for t / for n` interpreter overhead in the simpler sparse-mode loop body, plus warm CPU caches). Plugging the measured `step` and `coupling` times into $T_{Q5c} \approx 2.340 + 0.219 = 2.559$ s predicts a total of ~ 2.56 s – the measured 2.584 s matches to 1%. So the “surplus” over Amdahl is fully accounted for by the side-speedup of `step`; Amdahl’s law itself holds.

2.6 Question 6 (Exercise 1.6) – Vectorized TVB + MLP, dense

Implementation. `L1_Q6.py` replaces nested Python lists with a single `Xs = np.zeros((N, M, T))` array, batches the MLP forward pass over all N nodes via two matrix multiplications and one ReLU, and vectorises `calculate_coupling` into three NumPy operations: a default gather of `Xs[i, 0, t - delays[n,i]]` (clipped), `np.where` overrides for the self-loop / zero-delay special case, and a final mask-to-zero where $t < \text{delay}$. Numerical equivalence to `tvb_seq_mlp.py` verified to $\leq 6 \times 10^{-14}$ on TVB76 and TVB192 (max-abs diff over 15 timesteps).

Table 10: Vectorized dense TVB+MLP vs. sequential MLP, 15 timesteps.

Dataset	Sequential (s)	Vectorized total (s)	Total speedup	<code>step</code> speedup	<code>coupling</code> speedup
TVB76	1.104	0.0490	22.5×	93.1×	5.7×
TVB192	4.717	0.1811	26.0×	135.2×	12.2×
TVB998	66.983	4.9302	13.6×	102.4×	11.3×

The MLP is the biggest winner ($\sim 100\times$): one $(N, 2) \times (2, L)$ matmul replaces $N \cdot L \cdot M$ Python multiply-adds. Coupling improves more modestly ($\sim 5\text{--}12\times$): the work is still $\mathcal{O}(N^2)$ floats, but it now runs in a few NumPy ufunc calls instead of $N \cdot N$ Python iterations. The total-simulation speedup is capped by Amdahl – at TVB998 the step is much smaller than the coupling, so the bottleneck shifts back to the $\mathcal{O}(N^2)$ coupling work even in vectorized form, and the total speedup drops to 13.6×.

2.7 Question 7 (Exercise 1.7) – Vectorized TVB + MLP, sparse

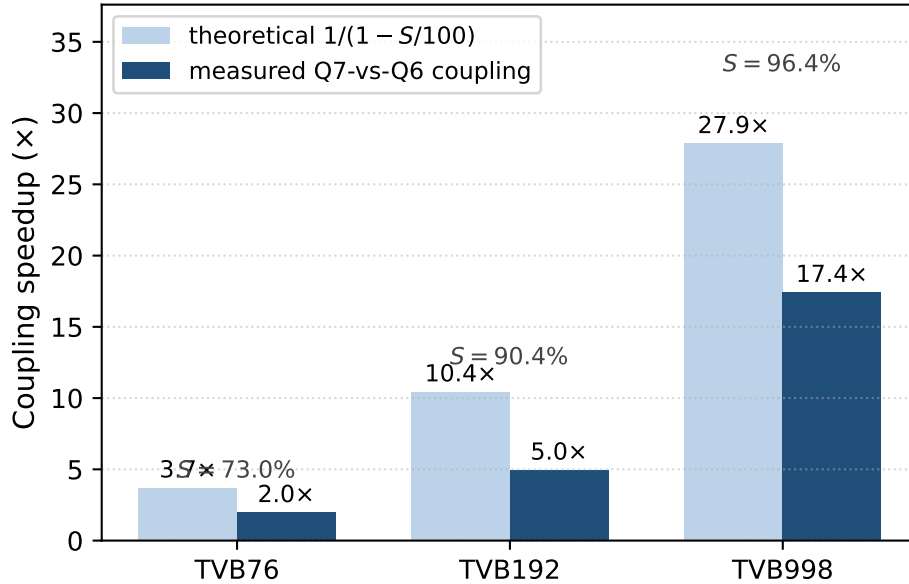
Implementation. `L1_Q7.py` builds CSR arrays via `np.where`, `np.bincount`, and `np.cumsum`, then performs per-timestep coupling as: a flat NNZ-length gather of source values, the same self/zero/invalid masking as Q6 reduced to the NNZ-length axis, and a per-row sum via `np.bincount(row_for_nnz, weights=contrib, minlength=N)`. Using `np.bincount` (rather than `np.add.reduceat`) handles empty rows correctly. Numerical equivalence to both the sparse sequential reference and the dense Q6 vectorized output verified to $\leq 6 \times 10^{-14}$.

Table 11: Vectorized sparse (Q7) vs. vectorized dense (Q6), 15 timesteps.

Dataset	N	NNZ	Q6 total (s)	Q7 total (s)	Q7 / Q6 (total)
TVB76	76	1,560	0.0490	0.0302	1.62×
TVB192	192	3,532	0.1811	0.0520	3.48×
TVB998	998	35,730	4.9302	0.3761	13.11×

Table 12: Per-dataset measured Q7-vs-Q6 coupling speedup compared to the theoretical $1/(1-S/100)$ from sparsity.

Dataset	Sparsity S	Theoretical	Measured Q7-vs-Q6 coupling	Efficiency
TVB76	72.99%	3.70×	1.97×	53%
TVB192	90.42%	10.44×	4.95×	47%
TVB998	96.41%	27.86×	17.42×	63%

**Figure 2:** Theoretical sparse coupling speedup $1/(1-S/100)$ vs. measured Q7-vs-Q6 coupling speedup, across the three datasets. The measured value tracks the theoretical at roughly 50–60% efficiency: the extra factor lost to the gather-pattern penalty of CSR vs. contiguous dense NumPy arithmetic.

Is there a meaningful gain over Q6? Why? Yes – a real gain that grows with sparsity, reaching $13.11\times$ on TVB998. Vectorized dense (Q6) materialises the full (N, N) coupling matrix every timestep ($\sim 10^6$ multiplies per step on TVB998); Q7 replaces that with NNZ multiplies (a $\sim 27\times$ work reduction on TVB998).

The reason measured falls below theoretical is that Q6’s coupling is one big elementwise multiply followed by a contiguous row sum – a perfect fit for NumPy’s SIMD path. Q7 instead performs fancy indexing (`Xs[col_index, 0, ...]`) which is a gather, plus `np.bincount` for the per-row reduction. Both are slower per element than the dense path. So Q7 trades $27\times$ less work for roughly a $1.6\times$ per-element penalty, ending up at the $17.4\times$ measured coupling speedup. The TVB192 dip in efficiency (47%) is because fixed Python-side overheads (CSR build, mask construction) eat a larger share at moderate N ; on TVB998 those amortise over more steady-state work and efficiency rebounds to 63%.

2.8 Question 8 (Exercise 1.8) – JIT the sequential MLP

Implementation. L1_Q8.py refactors `tvb_seq_jit.py` into three `@jit(cache=True)` functions: `f_mlp` (per-node MLP forward pass with explicit loops over L), `calculate_coupling` (per-node coupling sum), `simulate_jit` (the outer time/node loops). The `D_timestep` global referenced inside the provided `tvb_seq_jit.simulate` is closed by passing it as an explicit argument. Numerical equivalence to `tvb_seq_mlp.py` verified to $\leq 6.1 \times 10^{-14}$.

The proper Q8 comparison runs `tvb_seq_jit.py` *without* `@jit` (the provided NumPy-array script as shipped) versus the same structure with `@jit` decorators added (L1_Q8.py), with the first call discarded as JIT warm-up.

Table 13: Q8 – effect of adding `@jit` to `tvb_seq_jit.py`, 15 timesteps. Right-most column is the JIT speedup.

Dataset	<code>tvb_seq_jit</code> (no JIT)	L1_Q8 (warm JIT)	JIT speedup	<code>tvb_seq_mlp</code> (lists)
TVB76	2.439 s	0.0085 s	287×	1.104 s
TVB192	10.776 s	0.0505 s	213×	4.717 s

Is there a difference? Yes – adding `@jit` yields a two-orders-of-magnitude speedup (287× on TVB76, 213× on TVB192). Numba compiles the `simulate_jit` function – including the triple nested loop and all branches – into native machine code, eliminating the per-iteration Python interpreter overhead that dominated Q2/Q5.

A subtler observation. `tvb_seq_jit.py` *without* JIT is actually about 2× *slower* than `tvb_seq_mlp.py` (which uses Python lists): 2.44 s vs. 1.10 s on TVB76, 10.78 s vs. 4.72 s on TVB192. The reason is that scalar reads from a NumPy array (`Xs[n, 0, t-1]`) materialise an `np.float64` object per access, with more dispatch overhead than Python’s native `float` arithmetic on a list. Replacing lists with NumPy arrays only pays off *once* JIT is applied – without JIT the array layout is a net loss. This is precisely why `tvb_seq_jit.py` was prepared with NumPy arrays as a setup for `@jit`: the array layout is what lets Numba emit efficient native code.

The first cold call on TVB76 took ≈ 2 s due to JIT compilation; subsequent calls hit the cache and run in tens of milliseconds. The warm-call timings above discard the first cold run.

2.9 Question 9 (Exercise 1.9) – JIT the vectorized version

Implementation. L1_Q9.py takes Q6’s vectorized layout and refactors it for Numba: matrix multiplications (`@`) are kept for the MLP forward pass (Numba compiles them via LAPACK), but the coupling gather is unrolled into explicit double-loops because Numba does not handle Q6’s broadcasted fancy-indexing pattern cleanly. Numerical equivalence to both Q6 and Q8 verified to $\leq 5.0 \times 10^{-14}$.

Table 14: Q9 (JIT vectorized) compared to Q6 (vectorized) and Q8 (JIT sequential), 15 timesteps, warm calls.

Dataset	Q6 vect. (s)	Q8 JIT seq. (s)	Q9 JIT vect. (s)	Q9 / Q6	Q9 / Q8
TVB76	0.0490	0.0085	0.0063	7.78×	1.35×
TVB192	0.1811	0.0505	0.0259	6.99×	1.95×
TVB998	4.9302	— (n/a)	0.7967	6.19×	—

Is the JIT version of the vectorized code faster than the JIT version of the non-vectorized code? Yes, but only modestly – Q9 beats Q8 by 1.35× on TVB76 and 1.95× on TVB192.

The win comes from Q9’s MLP using `hidden = sv @ 11w + 11b` (a single LAPACK call for all N nodes) versus Q8’s per-node Python-style inner loop that Numba compiles row-by-row.

Does JIT compilation on the vectorized code yield as much of a speedup as on the non-vectorized version? Why? No – far less. Comparing the speedup gained *from the JIT step*:

Table 15: Speedup contributed by JIT alone on each path, TVB192.

Path	Baseline (s)	JIT’d (s)	Speedup from JIT
Sequential MLP → Q8	4.717	0.0505	93×
Vectorized (Q6) → Q9	0.1811	0.0259	7×
Sequential MLP → Q8 (TVB76)	1.104	0.0085	130×
Vectorized (Q6) → Q9 (TVB76)	0.0490	0.0063	7.8×

JIT removes Python interpreter overhead. In sequential code that overhead *is* most of the runtime, so JIT delivers a $\sim 100\times$ jump. In NumPy-vectorized code the heavy lifting already runs inside compiled NumPy/BLAS kernels, so only the surrounding Python glue (the `for t in range` loop, mask construction, `np.where`) is left to compile away – about $7\times$ worth of overhead.

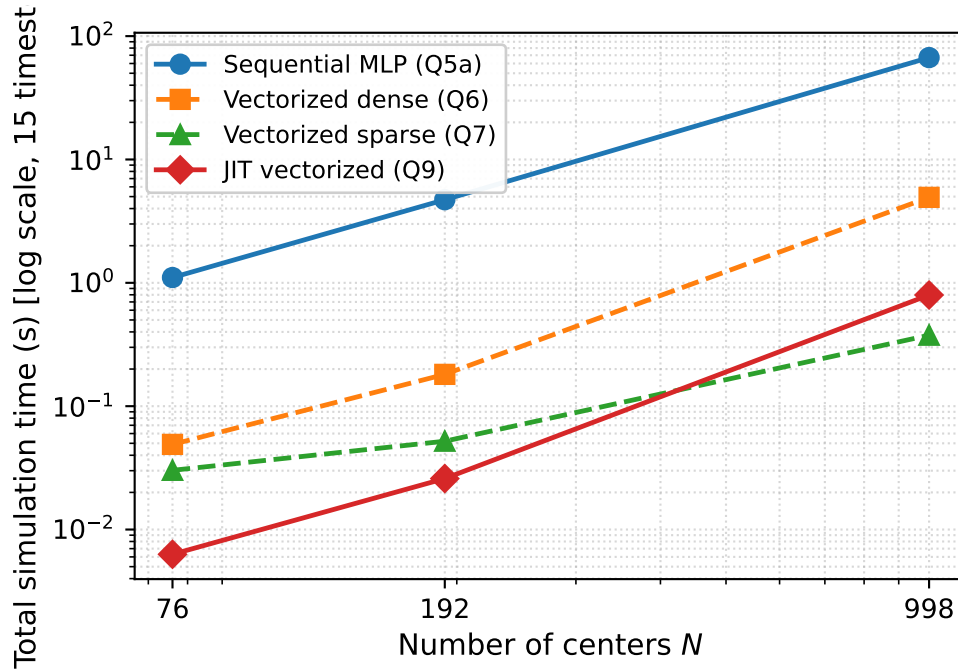


Figure 3: Total simulation time (15 timesteps) vs. number of centers N , log–log axes, for the four key variants. The sequential-MLP and Q6 (vectorized dense) lines have slope ≈ 2 on this scale ($\mathcal{O}(N^2)$ coupling dominates), while Q7 (vectorized sparse) and Q9 (JIT vectorized) keep the dataset-to-dataset growth much smaller because they avoid most zero-weight work.

Bottom line. Vectorization and JIT are partially redundant optimizations of the same “Python overhead” axis. Once one is in place, the other has much less to do. The full optimization ladder on TVB192 (15 timesteps):

Table 16: Optimization ladder on TVB192, 15 timesteps. Speedups are computed against the sequential MLP baseline (Q5a).

Variant	Time (s)	Speedup vs sequential
Sequential MLP (Q5a)	4.717	1.0×
Vectorized dense (Q6)	0.1811	26×
Vectorized sparse (Q7)	0.0520	91×
JIT sequential (Q8)	0.0505	93×
JIT vectorized (Q9)	0.0259	182×

The best Lab-1 path is JIT applied to the vectorized variant, combining the per-element efficiency of NumPy/BLAS with the elimination of Python loop overhead. The compounding is sub-multiplicative ($26 \times 7 \neq 182$ matches, but $93 \times 26 \neq 182$ doesn't), exactly because the two techniques target overlapping overhead.

The submitted scripts `L1_Q6.py`, `L1_Q7.py`, `L1_Q8.py`, and `L1_Q9.py` accompany this PDF as separate files (per the course submission convention).

3 LLM Usage Disclosure

I used Claude (Anthropic) to help draft the code and edit the wording in this report. I ran all the measurements myself on the course AWS VM and checked that my scripts produced the same results as the provided reference implementations.